

12306 项目面试题答案文档（按当前仓库源码定制）

| 适用对象：`mock\_interview\_questions\_v2.md`

| 使用原则：这份答案**以当前仓库源码为准**，优先保证“能对上代码、能对上压测产物、能对上表结构”。

| 面试时不要为了好听去硬编。遇到和简历表述不完全一致的地方，先澄清，再展开。

先说四个真实性底线

1. **这个仓库本质上是一个 Spring Boot 票务核心服务，不是独立的 Spring Cloud Gateway。

**

所以如果面试官问“你这个高性能网关是 Gateway 还是 Netty 手写”，建议先澄清：

“更准确说，我这个项目是把票务入口、防护、缓存、库存和 AI 助理都收敛在一个服务里，HTTP 入口由 Controller 承接，不是单独拆出来的 Spring Cloud Gateway 网关项目。”

2. **项目里存在两条购票链路。**

一条是 `/train/book/lua`，走 **RocketMQ 事务消息**；另一条是 `TicketTools.bookTicket()`，走 **本地事务直写**。

所以如果面试官追问“你简历里事务消息那条链路怎么做的”，你重点讲 `/train/book/lua + TicketTransactionListener + TicketConsumer` 这条。

3. **Sentinel 在源码里明确落地的是限流，不是完整的熔断降级体系。**

现在有接口级 QPS 规则和热点参数限流规则，但没有单独的 `DegradeRule`。

所以面试时可以说“当前项目已落地限流，熔断降级是下一步扩展方向”，不要硬说自己已经把超时熔断全配完了。

4. **AI 链路是混合栈，不是纯 reactive-only 应用。**

项目同时引入了 `spring-boot-starter-web` 和 `spring-boot-starter-webflux`。

我的表述建议是：

“传统票务接口还是 MVC 风格，AI 流式问答链路使用了 Reactor/Flux/Sinks 的响应式编排。”

模块零：破冰、真实性、抗压

1. 基本盘问

这一类没有源码可帮你，必须按你真实情况说。建议控制在 30 秒内：

| “我是 XX 届，XX 专业。操作系统、计算机网络、数据库和 Java
| 并发这些基础我一直在结合项目复盘。到岗时间这块按实际情况说，
| 如果需要我也可以尽快安排。”

不要背大段鸡汤，直接、正常、像一个工程师。

2. 项目真实性拷问

建议按下面这个版本说，但请把“负责范围”换成你真实做过的部分：

| “这个项目不是我只会跑一下 Demo 的那种。我是按源码完整做过、改过和压过的。
| 后端这块我主要做了四件事：
| 第一，高并发查票链路，做了 Caffeine + Redis 两级缓存、布隆过滤器和 Redisson 锁；
| 第二，库存扣减链路，做了 Redis Lua 原子扣减和分段库存；
| 第三，订单一致性链路，做了 RocketMQ 事务消息、退票补偿和库存对账；
| 第四，AI 助理链路，做了 LangChain4j、多 Agent 路由、Hybrid RAG、语义缓存、
| 双层记忆和 RAGAS 评估。”

如果面试官继续问“你具体改过哪些文件”，可以点这些：

- `src/main/java/com/ahu/ticket/controller/TicketController.java`
- `src/main/java/com/ahu/ticket/service/StockBucketService.java`
- `src/main/java/com/ahu/ticket/mq/TicketTransactionListener.java`
- `src/main/java/com/ahu/ticket/service/impl/TicketTools.java`
- `src/main/java/com/ahu/ticket/service/impl/RagServiceImpl.java`
- `src/main/java/com/ahu/ticket/rag/RedisChatMemoryStore.java`
- `src/main/java/com/ahu/ticket/rag/HybridParentChildContentRetriever.java`

3. 日常与抗压探底

别过度包装，核心是“表达稳定、能协作、可落地”。模板：

| “我选开发是因为我更喜欢把问题拆开、定位和落地成系统，不只是停留在讨论层面。
| 平时也会看源码、做压测、补基础。base 地按实际情况，我更看重业务密度和成长空间。”

项目一：高并发售票核心链路

1. “高性能微服务网关” “万级并发” “数据零偏差”

怎么答：

| “先澄清一下，这个仓库不是独立 Spring Cloud Gateway 网关，
| 更准确说是票务核心入口服务。用户请求先进 `TicketController`，查询链路会先过
| Caffeine 本地缓存，再过 Redisson 布隆过滤器，再查 Redis，最后才在 Redisson
| 分布式锁保护下回源 MySQL。
| 万级并发这块，我是用 JMeter 压的，仓库里保留了 `ticket-query-load-test.jmx`
| 和多份压测报告。默认单 worker 配置是 `1000 users / 120s ramp / 600s duration`。
| 2026-04-14 的稳定版查询报告里，`sampleCount=1,794,329`，`error=0`，吞吐约
| `5274.74 req/s`，平均响应时间约 `29.98ms`，p90/p95/p99 大约是 `192ms / 202ms /
| 268ms`。
| 数据零偏差不是靠感觉，是靠对账接口验证的。项目里有 `/audit/reconcile/stock`，公式是
| `expectedRedisStock = dbStock - pendingCount - processingCount`，同时还会校验所有
| bucket 的和必须等于理论库存。”

****补一句非常关键：****

| “仓库里也保留了一份 2026-04-14 的 10k 级压力报告，吞吐大概 `3402.68 req/s`，但有
| `0.47%` 错误率、长尾非常差，所以我不会把它当稳定态成绩去吹，我更认可 0
| 错误的稳定结果。”

2. Cache Aside + Caffeine + Redis + Bloom + DCL

****怎么答：****

| “我的查询顺序是 `Caffeine -> BloomFilter -> Redis -> MySQL`。
| Caffeine 是 JVM 内一级缓存，优先吃热点合法请求；BloomFilter 用来挡非法车次，避免
| Redis/DB 被恶意穿透；Redis 是二级分布式缓存；最后才在 `lock:query:{trainNumber}`
| 这个粒度的 Redisson 锁下回源数据库。
| 所谓 DCL 就是两次检查：第一次在加锁前先查缓存，第二次是拿到锁以后再查一次 Redis。
| 去掉第一重检查会导致大量请求都去抢锁，吞吐明显下降；
| 去掉第二重检查会导致第一个线程回填完缓存后，后续拿到锁的线程还会继续打 DB，
| 形成重复回源。
| Redis 和 MySQL 不一致的兜底，一方面靠 TTL 自然收敛，另一方面靠重新回源回填；
| 库存链路则另外做了对账和补偿，不靠缓存自愈硬扛。”

****如果被追问“DB QPS 降了多少”：****

建议答口径，不要编具体数：

| “源码里没有直接固化 A/B 对照数值，但从架构上讲，本地缓存命中会直接绕过 Redis 和
| DB，热点查询被拦在 JVM 内；如果要严谨给数字，我会基于 MySQL QPS、Redis 命中率和
| JMeter 分组报告做前后对比，不会口头硬报一个值。”

3. 布隆过滤器、误判率、增量更新

****怎么答：****

| “布隆过滤器是在查询链路里 Redis 之前做的，代码在 `TicketController.queryTrain()`。
| 初始化在 `/train/init`，会把合法车次写进 `bloom:train\_numbers`。
| 布隆过滤器只会有假阳性，不会有假阴性。也就是说它最多是把一部分非法请求放过去，
| 不会把真实存在的车次误杀。所以我的兜底仍然是 Redis 和 DB 查询。
| 车次新增这块，当前项目是手动预热：新车次写库后调用 `/train/init` 同步库存和 Bloom。
| 生产里我会再补一个增量刷新任务或 CDC 监听，把新增车次自动写入 Bloom。”

4. Redis Lua、分段库存、`@Transactional`

****怎么答：****

| “我用 Lua 的核心原因不是单纯 `decr`，而是我要同时维护总库存 Key 和桶库存 Key，
| 两次操作必须是原子的。Pipeline 只是批量发送，不保证原子；单个 `decr` 只能改一个
| Key，不够。
| 分段库存的实现是在 `StockBucketService`，默认把总库存拆成 16 个 bucket。请求先按
| `orderSn hash` 命中一个起始桶，如果这个桶没票，不是直接返回无票，
| 而是线性探测后续桶，直到找到有票的桶或者全部桶都没票。所以不会出现 ‘第 5
| 桶没票但第 6 桶有票却被误判无票’ 的问题。
| `@Transactional` 只保证数据库事务，不会回滚 Redis。
| 这个项目里退票流程的处理方式是：MySQL 先事务性更新订单状态和 `t\_train.stock`，
| 然后发 RocketMQ 补偿消息异步回补 Redis。也就是说这里追求的是 ‘数据库强一致 + Redis
| 最终一致’。”

****如果问 `@Transactional` 失效场景：****

| “典型有三个：
| 第一，类内部自调用；
| 第二，方法不是 `public`；
| 第三，异常被吃掉或者不是 Spring 默认回滚的异常类型。
| 所以我会把事务边界放在对外暴露的方法上，不在内部随便绕代理。”

5. 并发单测、零超卖

****怎么答：****

| “我做了两层验证。
| 第一层是算法级并发测试，`TicketConcurrencyTest` 用 `CountDownLatch` 做发令枪，
| 200 线程同时抢 100 张票，断言成功数必须等于 100、失败数等于 100、最终库存精确为 0，

| 验证的是‘核心扣减逻辑不超卖’。

| 第二层是全链路压测和对账，前者验证吞吐和稳定性，后者通过`/audit/reconcile/stock`

| 检查 Redis 总库存、分桶库存和数据库状态是否一致。

| 所以我说的零超卖，不是只看单测，也不是只看接口返回，而是算法验证加账务校验一起做。

| ”

6. RocketMQ 事务消息三阶段

怎么答：

| “这条链路是`/train/book/lua`。第一步，Producer 发送 Half 消息到 Broker，Broker

| 先暂存，不投递给消费者。

| 第二步，Broker 回调`TicketTransactionListener.executeLocalTransaction()`，

| 这里我做两件本地事务动作：先调用`StockBucketService.reserve()`走 Redis Lua

| 原子预扣减，再往`t\_order`插一条`CREATED`状态的预订单。

| 如果这两步都成功，就返回`COMMIT`，Broker 才会把消息投递给`TicketConsumer`

| 去异步扣减 MySQL 真库存并把订单推进到`PENDING`；如果失败就`ROLLBACK`。

| 第三步，如果服务在本地事务执行后宕机，Broker 收不到明确提交状态，就会触发

| `checkLocalTransaction()`回查。我这里是查`t\_order`是否存在该`orderSn`，存在就

| `COMMIT`，不存在就`ROLLBACK`。”

7. `UNKNOWN`、事务悬挂、死信清理

怎么答：

| “当前代码里，如果回查时数据库查询异常，会返回`UNKNOWN`，让 Broker

| 下次继续回查。这能兜住短时抖动，但如果长期一直`UNKNOWN`，生产上不能无限挂着。

| 我的处理思路是三层：

| 第一，回查次数设上限；

| 第二，超过阈值后落监控告警和人工补偿队列；

| 第三，定时扫描`CREATED/PROCESSING`的异常订单，结合 Redis

| 预扣减映射做补单或回补。

| 也就是说，当前项目已经有回查兜底，进一步工程化会加‘最大回查次数 + 补偿工单 +

| 定时清理’。”

8. Sentinel、统一异常、线程池

怎么答：

| “Sentinel 现在落地的是两类规则：

| 一类是接口级 QPS 限流，比如`queryTrain`和`bookTicket`；

| 一类是热点参数限流，按第一个参数也就是`trainNumber`做车次级限流。
| 底层是 Sentinel 的滑动时间窗统计，不是我自己实现的令牌桶。
| 全局异常这块我用了`@RestControllerAdvice`和统一返回体`Result<T>`，
| 像缺参异常会返回 400，兜底异常会返回 500，并携带`traceId`方便排查。
| 线程池这块我专门配了守护线程池：`core=4, max=10, queue=100, CallerRunsPolicy`。
| 原因是`new Thread()`或`Executors.newFixedThreadPool()`在高并发下要么线程太多、
| 栈内存爆掉，要么无界队列把堆顶满。`daemon=true`的意思不是系统守护进程，而是
| JVM 退出时这些线程不会阻塞应用停机。”

****如果被追问 Sentinel 的算法：****

| “我的项目里用的是 QPS 规则和热点参数规则；如果继续深挖底层，我会讲 Sentinel 通过
| `LeapArray`把时间窗切成多个 bucket，每个 bucket 记录通过数、拒绝数等统计，
| 窗口滑动时复用和淘汰旧 bucket，这样比固定窗口更平滑，不会有明显的临界突刺。”

项目二：智能数字员工 / Reactive / RAG

1. 为什么做响应式系统，核心数据流怎么走

****怎么答：****

| “我做响应式的核心原因不是为了追新，而是 AI 问答天然有长连接、流式返回和大量 I/O
| 等待，如果还是传统一次性等待模型全答完再返回，首字节延迟和线程占用都会更差。
| 当前项目是混合架构：普通票务接口偏 MVC，AI 流式接口走`ReactiveRagController`->
| `RagServiceImpl.askQuestionFlux()`。
| 具体链路是：前端发起 SSE 请求后，先做 Guardrails 安全检查，
| 再尝试本地快路径工具执行，然后走 Query Router 做意图分类；非动作类问题会做 Query
| Refine，再走 embedding、语义缓存、Hybrid RAG 检索、多专家分发或 Agent 工具调用，
| 最后通过`Flux` / `Sinks.Many`按 token 推给前端。”

2. 防攻击具体做了什么

****怎么答：****

| “我这边防的主要是两类攻击。
| 第一类是票务系统的流量攻击，查询和购票入口通过 Sentinel 做接口级和热点参数限流，
| 查询侧还有 BloomFilter、防击穿缓存和分布式锁。
| 第二类是大模型 Prompt Injection。我在大模型入口前加了`checkGuardrails()`，
| 会对‘忽略之前指令’‘显示系统提示词’‘developer mode’这类关键词做黑名单拦截；
| 进入模型前还会把用户问题包在`<user\_input>`标签里，并补一层最高优先级安全指令，

| 禁止用户输入覆盖系统角色。”

****如果被追问“黑名单不够怎么办”：****

| “对，黑名单不是银弹，所以我做的是分层防护：入口黑名单先物理阻断明显攻击，Prompt
| 再做 delimiter 隔离，执行层再把工具权限限制到票务域内，避免模型拿到超范围能力。”

3. 为什么多专家协作，不用一个超大 Prompt

****怎么答：****

| “一个大 Prompt 可以做简单场景，但复杂场景容易互相干扰。
| 这个项目里我把能力拆成三类：
| `TicketExpert` 只负责查票、买票、退票、查订单，拿的是工具；
| `PolicyExpert` 只负责规章问答，拿的是 RAG；
| `GeneralExpert` 只负责闲聊。
| 前面再放一个 `Router` 做意图路由。这样做好处是：工具调用不会被政策长文本干扰，RAG
| 也不会被闲聊污染，职责边界更清晰，调优成本更低。”

4. 多 Agent 记忆污染、怎么识别、怎么隔离

****怎么答：****

| “多 Agent 的记忆污染，本质上是不同职责的上下文串味。比如上一轮在聊报销规则，
| 下一轮突然去买票，如果还共用同一份上下文，很容易让购票 Agent
| 带着无关政策内容做决策。
| 我这里做了两层处理。
| 第一层是 Session 前缀隔离：同一个用户同一个业务会话，会拆成
| `TICKET:username::sessionId`、`POLICY:username::sessionId`、
| `GENERAL:username::sessionId`，让专家各自维护自己的记忆空间。
| 第二层是实体桥接：我用 `EntityExtractor` 从当前输入里抽取车次、日期、出发地、
| 目的地、订单号，再把这些实体拼成 `contextBridge` 注入给目标专家。
| 这样做到的是‘记忆隔离，但业务事实不断裂’。
| 另外我还做了回放评估接口 `replayWithAndWithoutMemory()`，用答案差异比
| `diffRatio` 和 `0.45` 阈值去识别潜在记忆污染。”

5. SSE、分块推送、为什么比 MVC 更适合高并发

****怎么答：****

| “前端不是等模型全答完再一次性返回，而是服务端拿到 token 后就立刻切块下发，
| 协议上走的是 `text/event-stream`，也就是 SSE。

| 在 `askQuestionFlux()` 里，我把 LangChain4j 的 `TokenStream` 转成 Reactor 的
| `Flux<String>`；在 `toFlux()` 里通过 `Sinks.many().unicast().onBackpressureBuffer()`
| 把 token 持续发给前端。
| 相比传统 MVC 的同步阻塞模型，这种方式更适合 I/O 等待长、连接多、
| 单次响应持续时间长的场景。
| 但我也会诚实说：当前项目是混合栈，不是整个应用都跑在纯 WebFlux/Netty 上。
| 更准确的表述是 ‘AI 流式链路用 Reactor 进行响应式编排’ 。”

6. 背压、慢客户端、`Sinks.Many`

怎么答：

| “当模型生成速度快于客户端消费速度时，风险是服务端缓冲区不断积压，最终吃内存。
| 我这里用的是 `Sinks.many().unicast().onBackpressureBuffer()`，
| 它至少把推送和消费解耦了，避免生产者直接阻塞消费者。
| 不过我也会说明当前实现的短板：这里是 buffer 模式，但没有显式容量上限，
| 所以如果是特别慢的客户端，生产上我还会继续加最大缓冲、超时断开和连接数限制，
| 避免无限堆积。”

7. 语义缓存怎么判断等价，怎么防污染

怎么答：

| “语义缓存的实现是在 `RagServiceImpl`。我会先把问题做 embedding，然后遍历 Redis
| Hash 里的历史问答向量，计算余弦相似度，阈值是 `0.95`。
| 只有高相似问题才复用缓存答案。
| 之所以把阈值定得这么高，是因为票务场景对误命中很敏感。0.8
| 这种阈值可能把 ‘改签规则’ 和 ‘退票规则’、‘今天’ 和 ‘明天’、
| ‘北京到上海’ 和 ‘上海到北京’ 这类问题错判成相似，直接返回错答案。
| 为了防缓存污染，我在写缓存前还做了异常文本拦截，
| 只要答案里出现 ‘速率限制’ ‘限流’ ‘余额不足’ ‘Too Many Requests’ 这些字样，
| 就禁止写入缓存。”

8. Parent-Child、Milvus + BM25 + RRF + Reranker

怎么答：

| “长文档我没有整本直接喂给模型，而是先做 Parent-Child 双层切片。Parent 大块大约
| `1500/200 overlap`，保留完整上下文；Child 小块大约 `400/80 overlap`，
| 用于精准检索。
| 检索时先走两路召回：

| 一路 Dense，用 embedding 去 Milvus 查语义相近的 child；
| 一路 Sparse，用 BM25 查关键词命中的 child。
| 然后按 `parent\_id` 聚合，用 RRF 做融合排序，公式是 $\text{score}(d) = \sum 1 / (60 + \text{rank}_i(d))$ 。
| RRF 的价值在于 Dense 和 BM25 分数不可直接比较，但排名可以融合。
| 最后再交给 `ZhipuReranker` 做一层点对点精排，相当于粗排扩大召回、精排过滤噪声，
| 把最干净的 parent 上下文送给大模型。”

****如果问“整本规章的汇总怎么办”：****

| “当前实现最终返回 Top-K parent 块，不是把整本书一次性塞进去。对于跨章节汇总问题，
| 本质上靠多块 parent 召回和模型总结；如果以后要做更强的全章级总结，我会增加
| chapter-level 索引层。”

9. 幻觉治理、提示词边界、模型选型

****怎么答：****

| “我的幻觉治理不是只靠一句‘请不要胡编’，而是做了三层：
| 第一，检索层提高上下文质量，用 Parent-Child、Hybrid Search、RRF 和 Reranker
| 把错误上下文尽量挡在模型外；
| 第二，提示词层卡边界，比如在政策问答严格模式里明确要求‘只能依据参考资料作答，
| 资料不足就输出固定未知答案’；
| 第三，评估层用 RAGAS 回头量化 Faithfulness 和 Context Recall。
| 模型选型上，当前项目默认聊天模型是 `glm-4.5-air`，Judge 用 `glm-4-plus`。我选 GLM
| 的原因主要是中文效果、接入成本和时延/成本比比较平衡，
| 尤其是中文票务场景的工具调用和规章问答都比较顺手。”

10. 双层记忆、摘要压缩、关键参数不丢

****怎么答：****

| “我没有把所有历史对话无脑塞进上下文，而是做了双层记忆。
| 短期记忆用 `MessageWindowChatMemory` 保留最近 10 条精确消息，存在 Redis；
| 长期记忆在窗口发生淘汰时，才调用大模型把旧消息压成摘要，存到 MySQL 的
| `t\_chat\_summary`。
| 为了避免关键业务参数被摘要丢掉，我在摘要提示词里明确要求保留车次、日期、
| 站点等核心实体；另外像订单号、车次号、日期这类真正关键字段，我没有完全依赖摘要，
| 而是在主流程里用正则抽取和 Redis pending 状态单独保存。
| 所以我的设计不是‘全靠摘要’，而是‘短期精确保真 + 长期摘要降本 +
| 关键实体额外落状态’。”

11. RAGAS 怎么落地，指标怎么解释

****怎么答：****

| “我不是凭感觉调 RAG，而是专门做了评估接口和 Python 评估脚本。
| 在 2026-04-15 的重跑报告里，数据集大小是 25，Judge 模型是 `glm-4-plus`，最终
| `Faithfulness=0.8787`、`Context Precision=0.78`、`Context Recall=0.8667`。
| 对比 2026-04-01 的旧报告，Faithfulness 只有 `0.6727`，
| 说明后来这版在忠实度上有明显提升。
| 这里面：
| `Context Recall` 看的是参考答案所需的关键信息，检索上下文是否找回来；
| `Faithfulness` 看的是生成答案是否真的建立在检索上下文上，而不是模型自己脑补。
| 我会根据低分 case 反向优化。比如报告里‘候补购票是什么意思’这种问题是 0 分，
| 就说明知识库覆盖或召回链路有缺口；而‘提前两天退票要扣多少手续费’ Faithfulness
| 只有 0.5，则说明回答时丢了边界条件，要继续强化上下文完整性和回答模板约束。”

****顺手可以补一句：****

| “目前 `answer\_relevancy` 还是偏低，说明回答虽然更忠实了，但表达不够贴题，
| 这是下一步重点，不会把一个分数高了就说系统完美。”

模块三：精选高频深挖

1. AQS / ReentrantLock 底层

| “ReentrantLock 底层依赖 AQS，AQS 维护的是一个双向队列，本质上是 CLH 的变体。
| 线程抢锁失败后不会原地忙等，而是封装成 Node 挂到队列尾部，
| 前驱释放锁后再按规则唤醒后继。这样做的核心价值是把竞争线程组织起来，
| 避免无限自旋把 CPU 打满。公平锁和非公平锁的差别，本质就是新线程是否允许插队。”

2. CAP 怎么结合这个项目讲

| “购票场景我会拆开说。
| 对‘核心库存不能超卖’这件事，我偏向一致性，宁可让部分请求失败或重试，
| 也不能把库存卖成负数，所以 Redis 预扣减这里更像偏 CP 的思路。
| 但跨 Redis、MySQL、MQ 的整条链路，我接受最终一致，
| 也就是订单和缓存之间允许短时间不一致，再靠事务消息、补偿消息和对账收敛。
| 所以不是整个系统简单一句 AP 或 CP，而是‘核心扣减偏一致性，
| 跨存储编排接受最终一致性’。”

3. Bloom 不支持删除，怎么绕

- | “当前项目没有做动态删除，因为车次删除不是高频场景。
- | 如果线上真要避免全量重建，我会用三种办法之一：
- | 第一，版本化 Bloom，新老过滤器并存，平滑切换；
- | 第二，Bloom 只做正向放行，再叠加一个小型黑名单/墓碑集合作为逻辑删除层；
- | 第三，定时增量重建，把车次主数据快照批量刷新。
- | 也就是说，不会指望原生 Bloom 自己支持删除。”

4. Lua 脚本异常会不会执行一半

- | “不会。Redis 执行 Lua 脚本是单线程、原子执行的，
- | 同一个脚本中的多条命令要么整体执行完，要么在报错前不会被其他命令插入。
- | 就我们这种库存扣减脚本来说，不会出现总库存扣了但桶库存没扣、
- | 并且还被别的线程插进来的‘中间态可见’问题。”

5. Sentinel `LeapArray`

- | “`LeapArray` 可以理解成一个按时间片组织的循环统计数组。比如把 1 秒窗口切成多个
- | bucket，每个 bucket 统计通过数、异常数、RT 等；当前时间推进时，旧 bucket
- | 会被重置复用。相比固定窗口，它能减少窗口边界上的突刺问题，限流判断更平滑。”

6. 为什么做 `Orchestrator-Worker`

- | “这就是我项目里的 `Query Router + Experts`。
- | 路由员负责判断这是闲聊、票务动作还是规章问答；执行员只处理自己擅长的事情。
- | 这样设计有三个直接好处：
- | 第一，减少 prompt 互相污染；
- | 第二，工具权限更可控；
- | 第三，问题定位更清楚，哪条链路答错就修哪条。”

7. MVC 做 SSE 会不会拖垮线程

- | “如果是传统 Servlet 同步阻塞模型，长时间占着工作线程做流式返回，连接一多肯定危险。
- | 我项目里引入 Reactor/Flux 的目的，
- | 就是把这种持续推流场景从‘线程一直卡住等结果’改成‘异步接 token、按流推送’。
- | 不过我也会说明当前工程是混合栈，不是整个服务彻底切到纯 WebFlux，所以我讲的是‘AI
- | 流式链路响应式化’，不是‘整个系统已经彻底 Reactive 化’。”

8. 语义缓存阈值为什么是 0.95，不是 0.8

- | “因为这是票务系统，不是泛知识问答。
- | 阈值低了最严重的问题不是‘回答有点像’，而是**业务语义错位**。
- | 比如‘帮我退 G1001’和‘帮我买 G1001’，embedding 可能都很近；
- | ‘明天合肥到北京’和‘今天合肥到北京’也很近；
- | 但这类错命中会直接造成错误引导，甚至在工具调用场景里触发错误动作。
- | 所以我宁可少命中一些，也要把精度放前面，0.95 是更保守的工程阈值。”

最后给你的面试表达建议

1. 先澄清，再作答

这份项目里有几处简历表述很容易被问穿：

- “网关”其实更接近“票务核心入口服务”
- “事务消息链路”只对应其中一条购票路径
- “Sentinel 超时熔断”源码里主要落地的是限流
- “WebFlux”更准确是“AI 流式链路使用 Reactor 编排”

先澄清不会扣分，硬编才会。

2. 数字题只报有出处的

你目前可以放心报的数字：

- `2026-04-14` 稳定查询压测：`5274.74 req/s`，`0 error`，`1,794,329 samples`
- 平均响应时间约 `29.98ms`
- p90/p95/p99 约 `192/202/268 ms`
- `2026-04-15` RAG 评估：`Faithfulness=0.8787`，`Context Precision=0.78`，`Context Recall=0.8667`

3. 一定要把“我为什么这么设计”说出来

面试官真正想听的是取舍：

- 为什么要本地缓存再 Redis
- 为什么 Lua 而不是 `decr`
- 为什么要把库存拆 bucket
- 为什么 Redis 和 MySQL 不追求强一致而是最终一致
- 为什么要多 Agent 而不是一个大 Prompt
- 为什么语义缓存阈值要保守

你只要把这些“为什么”说顺，这个项目就不是背八股，而是工程设计。

一段 90 秒项目总述

| “我这个项目分两块。第一块是 12306 高并发票务链路，我主要做了查询防护、
| 库存扣减和一致性治理。查询入口用了 Caffeine、本地 Bloom、Redis 和 Redisson
| 锁做四层防护；库存扣减用 Redis Lua 和 16 桶分段库存做热点分散；下单链路里一条走
| RocketMQ 事务消息，另一条给 AI Agent 直连工具调用，退票则通过 MySQL 事务加 MQ
| 异步补偿保证最终一致，并且做了库存对账接口验证零偏差。
| 第二块是 AI 智能客服，我用 LangChain4j 做了 Query Router、多专家协作、Hybrid RAG、
| 语义缓存和双层记忆。检索层用了 Milvus + BM25 双路召回、RRF 融合和 Reranker 精排，
| 生成层做了 Prompt Injection 防护和工具调用，评估层引入了 RAGAS。
| 整体上我不是把模型简单接进来，而是把它做成了一个能查、能问、
| 能执行票务动作的工程化系统。”